LAPORAN UTS

ALGORITMA DAN PEMROGRAMAN LANJUT

Muhammad Alfauzi Syahputra	**2509106006**
Syarafina Nur Amalia	**2509106016**
Diftya Azzahra	**2509106042**

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

DAFTAR ISI

DAFTAR ISI **1**
1. Array & Array Multi Dimensi **2**
A. String 2
B. Array Statis 2
C. Array Dinamis 3
D. Array 1 Dimensi 3
E. Array Multi Dimensi (2 Dimensi) 4
F. Array Multi Dimensi (3 Dimensi) 4
G. Penjumlahan Matriks 5
H. Pengurangan Matriks 6
I. Perkalian Matriks 6
J. Transpose Matriks 7
K. Invers Matriks 7
2. Pointer & Struct **9**
A. Pointer 9
B. Struct 9
C. Array & Struct 10
D. Pointer & Struct 10
E. Union 11
3. Procedure & Fungsi **12**
A. Procedure 12
B. Fungsi 12
C. Parameter 13
D. Variabel Lokal & Global 13
E. Variabel Static 14
F. Fungsi Rekursif 14
4. Sorting **16**
A. Insertion Sort 16
B. Heap Sort 16
C. Merge Sort 17
D. Quick Sort 18
E. Tree Sort 19
F. Exchange Sort 21
G. Radix Sort 21
H. Shell Sort 22
I. Bubble Sort 23
J. Selection Sort 23
5. Searching **25**
A. Fibonacci Search 25
B. Binary Search 26
C. Interpolation Search 27
D. Sequential Search 28

1. Array & Array Multi Dimensi

A. String

String adalah kumpulan karakter yang digunakan untuk menyimpan teks dalam sebuah variabel. Dalam bahasa pemrograman C atau C++, string biasanya disimpan dalam bentuk array karakter yang diakhiri dengan karakter null (\0). String sering digunakan untuk menyimpan data berupa teks seperti nama, alamat, atau kalimat.

Contoh program:

```
#include <iostream>
#include <conio.h>
using namespace std;

int main() {
    char kata[6];
    kata[0] = 'H';
    kata[1] = 'e';
    kata[2] = 'b';
    kata[3] = 'a';
    kata[4] = 't';
    kata[5] = '\0';

    cout << "Isi string adalah : " << kata << endl;
}
```

B. Array Statis

Array statis adalah array yang ukuran elemennya sudah ditentukan saat program ditulis dan tidak dapat diubah selama program dijalankan. Semua elemen array disimpan secara berurutan di dalam memori.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int angka[5] = {10, 20, 30, 40, 50};

    for(int i = 0; i < 5; i++) {
        cout << angka[i] << endl;
    }

    return 0;
}
```

C. Array Dinamis

Array dinamis adalah array yang ukuran elemennya dapat ditentukan saat program sedang berjalan (runtime). Dalam C++, array dinamis biasanya dibuat menggunakan operator new untuk mengalokasikan memori dan delete untuk menghapusnya setelah tidak digunakan.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Masukkan jumlah data: ";
    cin >> n;

    int *data = new int[n];

    for(int i = 0; i < n; i++) {
        cout << "Data ke-" << i+1 << ": ";
        cin >> data[i];
    }

    cout << "Data yang dimasukkan:\n";
    for(int i = 0; i < n; i++) {
        cout << data[i] << endl;
    }

    delete[] data;
}
```

D. Array 1 Dimensi

Array satu dimensi adalah struktur data yang digunakan untuk menyimpan beberapa nilai dengan tipe data yang sama dalam satu baris indeks. Setiap elemen dalam array dapat diakses menggunakan nomor indeks yang dimulai dari 0.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int nilai[4];

    for(int i = 0; i < 4; i++) {
        cout << "Masukkan nilai: ";
        cin >> nilai[i];
```

```
    }

    cout << "Daftar nilai:\n";
    for(int i = 0; i < 4; i++) {
        cout << nilai[i] << endl;
    }

    return 0;
}
```

E. Array Multi Dimensi (2 Dimensi)

Array dua dimensi adalah array yang memiliki dua indeks, yaitu baris dan kolom. Struktur ini sering digunakan untuk menyimpan data dalam bentuk tabel atau matriks.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int matriks[2][3] = {
        {1,2,3},
        {4,5,6}
    };

    for(int i = 0; i < 2; i++) {
        for(int j = 0; j < 3; j++) {
            cout << matriks[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

F. Array Multi Dimensi (3 Dimensi)

Array tiga dimensi merupakan pengembangan dari array dua dimensi dengan tambahan satu dimensi lagi. Struktur ini dapat dibayangkan sebagai kumpulan beberapa matriks yang disusun dalam satu variabel array.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int data[2][2][2] = {
```

```
        {{1,2},{3,4}},
        {{5,6},{7,8}}
    };

    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            for(int k=0;k<2;k++){
                cout << data[i][j][k] << " ";
            }
            cout << endl;
        }
        cout << endl;
    }

    return 0;
}
```

G. Penjumlahan Matriks

Penjumlahan matriks dilakukan dengan menjumlahkan elemen-elemen yang berada pada posisi yang sama di kedua matriks.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int A[2][2] = {{1,2},{3,4}};
    int B[2][2] = {{5,6},{7,8}};
    int C[2][2];

    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    cout<<"Hasil Penjumlahan Matriks:\n";
    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            cout<<C[i][j]<<" ";
        }
        cout<<endl;
    }
}
```

H. Pengurangan Matriks

Pengurangan matriks dilakukan dengan mengurangkan elemen-elemen yang berada pada posisi yang sama pada kedua matriks.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int A[2][2] = {{5,6},{7,8}};
    int B[2][2] = {{1,2},{3,4}};
    int C[2][2];

    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            C[i][j] = A[i][j] - B[i][j];
        }
    }

    cout<<"Hasil Pengurangan Matriks:\n";
    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            cout<<C[i][j]<<" ";
        }
        cout<<endl;
    }
}
```

I. Perkalian Matriks

Perkalian matriks dilakukan dengan mengalikan baris dari matriks pertama dengan kolom dari matriks kedua kemudian menjumlahkannya.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int A[2][2]={{1,2},{3,4}};
    int B[2][2]={{5,6},{7,8}};
    int C[2][2];

    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            C[i][j]=0;
```

```
            for(int k=0;k<2;k++){
                C[i][j]+=A[i][k]*B[k][j];
            }
        }
    }

    cout<<"Hasil Perkalian Matriks:\n";
    for(int i=0;i<2;i++){
        for(int j=0;j<2;j++){
            cout<<C[i][j]<<" ";
        }
        cout<<endl;
    }
}
```

J. Transpose Matriks

Transpose matriks adalah proses menukar posisi baris menjadi kolom.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int A[2][3]={{1,2,3},{4,5,6}};

    cout<<"Transpose Matriks:\n";

    for(int i=0;i<3;i++){
        for(int j=0;j<2;j++){
            cout<<A[j][i]<<" ";
        }
        cout<<endl;
    }
}
```

K. Invers Matriks

Invers matriks adalah matriks yang jika dikalikan dengan matriks asal akan menghasilkan matriks identitas. Invers biasanya hanya dapat dilakukan pada matriks persegi.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    float A[2][2]={{4,7},{2,6}};
```

```
    float det;

    det = (A[0][0]*A[1][1]) - (A[0][1]*A[1][0]);

    cout<<"Invers Matriks:\n";

    cout<<A[1][1]/det<<" "<<-A[0][1]/det<<endl;
    cout<<-A[1][0]/det<<" "<<A[0][0]/det<<endl;
}
```

2. Pointer & Struct

A. Pointer

Pointer adalah variabel yang digunakan untuk menyimpan alamat memori dari variabel lain. Dengan menggunakan pointer, program dapat mengakses atau memanipulasi data melalui alamat memori tersebut. Pointer biasanya digunakan dalam pengelolaan memori, pengiriman data ke fungsi, serta dalam struktur data seperti linked list dan tree.

Contoh program:

```
#include <iostream>
using namespace std;

int main() {
    int angka = 10;
    int *ptr;

    ptr = &angka;

    cout << "Nilai angka: " << angka << endl;
    cout << "Alamat angka: " << &angka << endl;
    cout << "Nilai pointer: " << *ptr << endl;

    return 0;
}
```

B. Struct

Struct (structure) adalah tipe data yang digunakan untuk menggabungkan beberapa variabel yang memiliki tipe data berbeda dalam satu kesatuan. Struct sering digunakan untuk merepresentasikan data yang lebih kompleks, seperti data mahasiswa, buku, atau produk.

Contoh program:

```
#include <iostream>
using namespace std;

struct Mahasiswa {
    string nama;
    int umur;
};

int main() {
    Mahasiswa mhs;

    mhs.nama = "Andi";
    mhs.umur = 20;
```

```
    cout << "Nama: " << mhs.nama << endl;
    cout << "Umur: " << mhs.umur << endl;

    return 0;
}
```

C. Array & Struct

Struct juga dapat digunakan dalam bentuk array untuk menyimpan banyak data dengan struktur yang sama. Dengan menggunakan array struct, program dapat menyimpan beberapa data yang memiliki tipe yang sama secara terorganisir.

Contoh program:

```
#include <iostream>
using namespace std;

struct Mahasiswa {
    string nama;
    int nilai;
};

int main() {
    Mahasiswa data[2];

    for(int i=0;i<2;i++){
        cout<<"Nama: ";
        cin>>data[i].nama;

        cout<<"Nilai: ";
        cin>>data[i].nilai;
    }

    for(int i=0;i<2;i++){
        cout<<data[i].nama<<" - "<<data[i].nilai<<endl;
    }
}
```

D. Pointer & Struct

Pointer juga dapat digunakan untuk mengakses atau memanipulasi anggota struct. Untuk mengakses anggota struct melalui pointer digunakan operator ->.

Contoh program:

```
#include <iostream>
using namespace std;

struct Buku {
```

```
    string judul;
    int harga;
};

int main() {
    Buku b1;
    Buku *ptr;

    ptr = &b1;

    ptr->judul = "Algoritma";
    ptr->harga = 50000;

    cout << ptr->judul << endl;
    cout << ptr->harga << endl;
}
```

E. Union

Union adalah tipe data yang memungkinkan beberapa variabel berbagi lokasi memori yang sama. Artinya, semua anggota union menggunakan ruang memori yang sama sehingga hanya satu nilai yang dapat disimpan pada satu waktu.

Contoh program:

```
#include <iostream>
using namespace std;

union Data {
    int angka;
    float nilai;
};

int main() {
    Data d;

    d.angka = 10;
    cout << "Angka: " << d.angka << endl;

    d.nilai = 3.14;
    cout << "Nilai: " << d.nilai << endl;
}
```

3. Procedure & Fungsi

A. Procedure

Procedure adalah bagian dari program yang digunakan untuk menjalankan suatu tugas tertentu tetapi tidak mengembalikan nilai kepada program yang memanggilnya. Dalam bahasa pemrograman C++, procedure biasanya ditulis menggunakan tipe data void. Penggunaan procedure membantu membuat program menjadi lebih terstruktur dan mudah dipahami karena program yang panjang dapat dibagi menjadi beberapa bagian kecil yang memiliki fungsi masing-masing.

Contoh program:

```
#include <iostream>
using namespace std;

void tampilPesan(){
    cout << "Selamat Belajar Pemrograman!" << endl;
}

int main(){
    tampilPesan();
    return 0;
}
```

B. Fungsi

Fungsi adalah blok program yang digunakan untuk melakukan suatu proses tertentu dan mengembalikan nilai (return value) kepada program yang memanggilnya. Fungsi biasanya digunakan ketika hasil dari suatu proses ingin digunakan kembali dalam bagian lain dari program. Dengan menggunakan fungsi, program menjadi lebih efisien, terstruktur, dan mudah dipelihara.

Contoh program:

```
#include <iostream>
using namespace std;

int tambah(int a, int b){
    return a + b;
}

int main(){
    int hasil;

    hasil = tambah(5,3);

    cout << "Hasil penjumlahan: " << hasil << endl;

    return 0;
```

```
}
```

C. Parameter

Parameter adalah variabel yang digunakan untuk menerima nilai yang dikirimkan ketika suatu fungsi atau prosedur dipanggil. Parameter berfungsi sebagai perantara untuk mengirimkan data dari program utama ke dalam fungsi atau prosedur yang digunakan.

Parameter dibagi menjadi dua jenis yaitu:

1. Parameter Formal : Parameter yang dituliskan pada saat deklarasi fungsi atau prosedur.
2. Parameter Aktual : Nilai atau data yang diberikan saat fungsi atau prosedur dipanggil.

Contoh program:

```
#include <iostream>
using namespace std;

int kali(int x, int y){
    return x * y;
}

int main(){
    cout << kali(4,6);
}
```

D. Variabel Lokal & Global

Variabel global adalah variabel yang dideklarasikan di luar fungsi utama maupun fungsi lainnya sehingga variabel tersebut dapat digunakan oleh semua fungsi yang ada di dalam program.

Contoh program:

```
#include <iostream>
using namespace std;

int nilai = 10;

void tampil(){
    cout << nilai << endl;
}

int main(){
    tampil();
}
```

Variabel lokal adalah variabel yang dideklarasikan di dalam suatu fungsi sehingga hanya dapat digunakan pada fungsi tempat variabel tersebut dibuat. Variabel ini tidak dapat diakses oleh fungsi lain di dalam program.

Contoh program:

```
#include <iostream>
using namespace std;

void contoh(){
    int angka = 5;
    cout << angka;
}

int main(){
    contoh();
}
```

E. Variabel Static

Variabel static adalah variabel yang dideklarasikan dengan kata kunci static di dalam suatu fungsi. Variabel ini memiliki sifat khusus yaitu nilainya akan tetap tersimpan meskipun fungsi tersebut selesai dijalankan. Ketika fungsi dipanggil kembali, variabel static akan melanjutkan nilai terakhir yang telah disimpan sebelumnya.

Contoh program:

```
#include <iostream>
using namespace std;

void hitung(){
    static int nilai = 0;
    nilai++;
    cout << nilai << endl;
}

int main(){
    hitung();
    hitung();
    hitung();
}
```

F. Fungsi Rekursif

Fungsi rekursif adalah fungsi yang memanggil dirinya sendiri untuk menyelesaikan suatu permasalahan. Metode ini biasanya digunakan untuk menyelesaikan masalah yang memiliki pola berulang, seperti perhitungan faktorial, deret Fibonacci, atau algoritma tertentu.

Dalam penggunaan fungsi rekursif harus terdapat kondisi penghentian (base case) agar proses pemanggilan fungsi tidak berlangsung tanpa batas. Kondisi ini berfungsi untuk menghentikan proses rekursi ketika syarat tertentu telah terpenuhi.

Contoh program:

```
#include <iostream>
using namespace std;

int faktorial(int n){
    if(n == 1)
        return 1;
    else
        return n * faktorial(n-1);
}

int main(){
    cout << faktorial(5);
}
```

4. **Sorting**

Sorting adalah proses mengurutkan data berdasarkan urutan tertentu, misalnya dari nilai terkecil ke terbesar (ascending) atau terbesar ke terkecil (descending). Proses ini sering digunakan untuk mempermudah pencarian data dan pengolahan informasi.

A. Insertion Sort

Insertion Sort bekerja dengan cara menyisipkan elemen pada posisi yang benar di bagian array yang sudah terurut.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int data[5] = {7,4,5,2,9};

    for(int i=1;i<5;i++){
        int key = data[i];
        int j = i-1;

        while(j>=0 && data[j] > key){
            data[j+1] = data[j];
            j--;
        }

        data[j+1] = key;
    }

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

B. Heap Sort

Heap Sort merupakan algoritma pengurutan yang menggunakan konsep heap atau pohon biner untuk menyusun data sehingga proses pengurutan dapat dilakukan dengan lebih efisien.

Contoh program:

```
#include <iostream>
using namespace std;

void swap(int &a, int &b){
    int temp = a;
    a = b;
    b = temp;
}
```

```
void heapify(int arr[], int n, int i){
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if(left < n && arr[left] > arr[largest])
        largest = left;

    if(right < n && arr[right] > arr[largest])
        largest = right;

    if(largest != i){
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n){

    for(int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for(int i = n-1; i > 0; i--){
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main(){

    int data[5] = {4,10,3,5,1};

    heapSort(data,5);

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

C. Merge Sort

Merge Sort menggunakan metode divide and conquer yaitu membagi array menjadi beberapa bagian kecil kemudian menggabungkannya kembali dalam kondisi terurut.

Contoh program:

```
#include <iostream>
using namespace std;
```

```
void merge(int arr[], int l, int m, int r){
    int i=l, j=m+1, k=l;
    int temp[100];

    while(i<=m && j<=r){
        if(arr[i] < arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while(i<=m)
        temp[k++] = arr[i++];

    while(j<=r)
        temp[k++] = arr[j++];

    for(int x=l;x<=r;x++)
        arr[x] = temp[x];
}

void mergesort(int arr[], int l, int r){
    if(l<r){
        int m = (l+r)/2;
        mergesort(arr,l,m);
        mergesort(arr,m+1,r);
        merge(arr,l,m,r);
    }
}

int main(){
    int data[5]={8,3,7,4,2};

    mergesort(data,0,4);

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

D. Quick Sort

Quick Sort adalah algoritma pengurutan yang menggunakan metode divide and conquer, yaitu dengan memilih satu elemen sebagai pivot, kemudian membagi elemen lain menjadi dua bagian: yang lebih kecil dari pivot dan yang lebih besar dari pivot.

Contoh program:

```
#include <iostream>
using namespace std;

void swap(int &a, int &b){
    int temp = a;
    a = b;
    b = temp;
}

int partition(int arr[], int low, int high){
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++){
        if(arr[j] < pivot){
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i+1], arr[high]);
    return i+1;
}

void quickSort(int arr[], int low, int high){
    if(low < high){
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi-1);
        quickSort(arr, pi+1, high);
    }
}

int main(){
    int data[5] = {9, 4, 6, 2, 8};

    quickSort(data,0,4);

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

E. Tree Sort

Tree Sort adalah algoritma pengurutan yang menggunakan struktur Binary Search Tree (BST). Data dimasukkan ke dalam tree, kemudian dilakukan inorder traversal untuk mendapatkan data yang sudah terurut.

Contoh program:

```
#include <iostream>
using namespace std;

struct Node{
    int data;
    Node* left;
    Node* right;
};

Node* newNode(int data){
    Node* node = new Node();
    node->data = data;
    node->left = node->right = NULL;
    return node;
}

Node* insert(Node* root, int data){
    if(root == NULL)
        return newNode(data);

    if(data < root->data)
        root->left = insert(root->left,data);
    else
        root->right = insert(root->right,data);

    return root;
}

void inorder(Node* root){
    if(root != NULL){
        inorder(root->left);
        cout<<root->data<<" ";
        inorder(root->right);
    }
}

int main(){
    int data[5] = {7,3,9,1,5};
    Node* root = NULL;

    for(int i=0;i<5;i++)
        root = insert(root,data[i]);

    inorder(root);
}
```

F. Exchange Sort

Exchange Sort adalah metode pengurutan yang dilakukan dengan membandingkan dua elemen dan menukarnya jika urutannya salah.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int data[5] = {8,3,6,2,7};

    for(int i=0;i<4;i++){
        for(int j=i+1;j<5;j++){
            if(data[i] > data[j]){
                int temp = data[i];
                data[i] = data[j];
                data[j] = temp;
            }
        }
    }

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

G. Radix Sort

Radix Sort adalah algoritma pengurutan yang bekerja dengan mengurutkan angka berdasarkan digitnya, mulai dari digit paling kecil hingga digit paling besar.

Contoh program:

```
#include <iostream>
using namespace std;

int getMax(int arr[], int n){
    int mx = arr[0];
    for(int i=1;i<n;i++)
        if(arr[i] > mx)
            mx = arr[i];
    return mx;
}

void countSort(int arr[], int n, int exp){
    int output[5];
    int count[10] = {0};

    for(int i=0;i<n;i++)
```

```
        count[(arr[i]/exp)%10]++;

    for(int i=1;i<10;i++)
        count[i] += count[i-1];

    for(int i=n-1;i>=0;i--){
        output[count[(arr[i]/exp)%10]-1] = arr[i];
        count[(arr[i]/exp)%10]--;
    }

    for(int i=0;i<n;i++)
        arr[i] = output[i];
}

void radixSort(int arr[], int n){
    int m = getMax(arr,n);

    for(int exp=1; m/exp>0; exp*=10)
        countSort(arr,n,exp);
}

int main(){
    int data[5]={170,45,75,90,802};

    radixSort(data,5);

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

H. Shell Sort

Shell Sort merupakan pengembangan dari Insertion Sort yang melakukan pengurutan dengan membandingkan elemen yang memiliki jarak tertentu (gap).

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int data[6] = {12,34,54,2,3,8};
    int n = 6;

    for(int gap = n/2; gap > 0; gap /= 2){
        for(int i = gap; i < n; i++){
            int temp = data[i];
            int j;
```

```
            for(j=i; j>=gap && data[j-gap] > temp; j-=gap)
                data[j] = data[j-gap];

            data[j] = temp;
        }
    }

    for(int i=0;i<n;i++)
        cout<<data[i]<<" ";
}
```

I. Bubble Sort

Bubble Sort adalah metode pengurutan yang bekerja dengan cara membandingkan dua elemen yang berdekatan dan menukarnya jika urutannya salah.

Contoh program:

```
#include <iostream>
using namespace std;

int main(){
    int data[5]={5,1,4,2,8};

    for(int i=0;i<4;i++){
        for(int j=0;j<4-i;j++){
            if(data[j] > data[j+1]){
                int temp = data[j];
                data[j] = data[j+1];
                data[j+1] = temp;
            }
        }
    }

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

J. Selection Sort

Selection Sort adalah metode pengurutan yang bekerja dengan cara mencari elemen terkecil dari array kemudian menukarnya dengan posisi pertama secara berulang.

Contoh program:

```
#include <iostream>
using namespace std;
```

```
int main(){
    int data[5] = {29,10,14,37,13};

    for(int i=0;i<4;i++){
        int min = i;

        for(int j=i+1;j<5;j++){
            if(data[j] < data[min])
                min = j;
        }

        int temp = data[i];
        data[i] = data[min];
        data[min] = temp;
    }

    for(int i=0;i<5;i++)
        cout<<data[i]<<" ";
}
```

5. Searching

Searching adalah proses dalam pemrograman yang digunakan untuk menemukan suatu data tertentu di dalam kumpulan data seperti array atau daftar nilai. Proses ini dilakukan dengan membandingkan data yang dicari dengan setiap elemen yang ada sampai data tersebut ditemukan atau dinyatakan tidak ada.

Dalam algoritma dan pemrograman, terdapat beberapa metode searching yang sering digunakan, seperti Sequential Search, Binary Search, Fibonacci Search, dan Interpolation Search, yang masing-masing memiliki cara kerja dan tingkat efisiensi yang berbeda.

A. Fibonacci Search

Fibonacci Search adalah metode pencarian pada array yang sudah terurut dengan menggunakan bilangan Fibonacci untuk menentukan posisi pembagian pencarian.

Metode ini mirip dengan Binary Search tetapi menggunakan perbandingan jarak berdasarkan deret Fibonacci.

Contoh program:

```
#include <iostream>
#include <algorithm>
using namespace std;

int fibonacciSearch(int arr[], int n, int x)
{
    int fib2 = 0;      // (m-2)'th Fibonacci
    int fib1 = 1;      // (m-1)'th Fibonacci
    int fib = fib2 + fib1; // m'th Fibonacci

    while (fib < n) {
        fib2 = fib1;
        fib1 = fib;
        fib = fib2 + fib1;
    }

    int offset = -1;

    while (fib > 1) {
        int i = min(offset + fib2, n - 1);

        if (arr[i] < x) {
            fib = fib1;
            fib1 = fib2;
            fib2 = fib - fib1;
            offset = i;
        }
        else if (arr[i] > x) {
            fib = fib2;
```

```
            fib1 = fib1 - fib2;
            fib2 = fib - fib1;
        }
        else
            return i;
    }

    if (fib1 && arr[offset + 1] == x)
        return offset + 1;

    return -1;
}

int main()
{
    int data[] = {5, 10, 15, 20, 25, 30, 35};
    int n = 7;
    int cari = 25;

    int hasil = fibonacciSearch(data, n, cari);

    if (hasil != -1)
        cout << "Data ditemukan pada indeks ke-" << hasil;
    else
        cout << "Data tidak ditemukan";

    return 0;
}
```

B. Binary Search

Binary Search adalah metode pencarian data pada array yang sudah terurut dengan cara membagi data menjadi dua bagian secara berulang sampai data yang dicari ditemukan atau tidak ada lagi bagian yang bisa diperiksa.

Algoritma ini lebih cepat dibandingkan pencarian berurutan karena tidak perlu memeriksa semua elemen satu per satu.

Contoh program:

```
#include <iostream>
using namespace std;

int main()
{
    int data[8] = {2, 5, 8, 12, 16, 23, 38, 56};
    int kiri = 0;
    int kanan = 7;
```

```
    int tengah;
    int cari;
    bool ditemukan = false;

    cout << "Masukkan angka yang dicari: ";
    cin >> cari;

    while (kiri <= kanan)
    {
        tengah = (kiri + kanan) / 2;

        if (data[tengah] == cari)
        {
            ditemukan = true;
            cout << "Data ditemukan pada indeks ke-" << tengah << endl;
            break;
        }
        else if (cari > data[tengah])
        {
            kiri = tengah + 1;
        }
        else
        {
            kanan = tengah - 1;
        }
    }

    if (!ditemukan)
    {
        cout << "Data tidak ditemukan dalam array." << endl;
    }

    return 0;
}
```

C. Interpolation Search

Interpolation Search adalah metode pencarian pada array yang sudah terurut dengan cara memperkirakan posisi data yang dicari berdasarkan nilai data tersebut. Algoritma ini bekerja lebih cepat jika data dalam array memiliki distribusi nilai yang relatif merata.

Berbeda dengan Binary Search yang selalu mengambil elemen tengah, Interpolation Search menghitung posisi kemungkinan dari nilai yang dicari menggunakan rumus tertentu.

Contoh program:

```
#include <iostream>
using namespace std;
```

```
int interpolationSearch(int arr[], int n, int x)
{
    int low = 0;
    int high = n - 1;

    while (low <= high && x >= arr[low] && x <= arr[high])
    {
        int pos = low + ((double)(high - low) / (arr[high] - arr[low])) * (x -
arr[low]);

        if (arr[pos] == x)
            return pos;

        if (arr[pos] < x)
            low = pos + 1;
        else
            high = pos - 1;
    }

    return -1;
}

int main()
{
    int data[] = {10, 20, 30, 40, 50, 60, 70};
    int n = 7;
    int cari = 50;

    int hasil = interpolationSearch(data, n, cari);

    if (hasil != -1)
        cout << "Data ditemukan pada indeks ke-" << hasil;
    else
        cout << "Data tidak ditemukan";

    return 0;
}
```

D. Sequential Search

Sequential Search adalah metode pencarian data yang dilakukan dengan cara memeriksa setiap elemen dalam array secara berurutan mulai dari elemen pertama hingga elemen terakhir sampai data yang dicari ditemukan atau seluruh data telah diperiksa.

Metode ini merupakan algoritma pencarian yang paling sederhana karena tidak memerlukan data dalam keadaan terurut.

Contoh program:

```
#include <iostream>
using namespace std;

int main()
{
    int data[6] = {14, 7, 25, 30, 18, 9};
    int cari;
    int posisi = -1;

    cout << "Masukkan nilai yang ingin dicari: ";
    cin >> cari;

    for(int i = 0; i < 6; i++)
    {
        if(data[i] == cari)
        {
            posisi = i;
            break;
        }
    }

    if(posisi != -1)
    {
        cout << "Data ditemukan pada indeks ke-" << posisi << endl;
    }
    else
    {
        cout << "Data tidak ditemukan dalam array." << endl;
    }

    return 0;
}
```